AWS Lambda Deep Dive

Explore AWS Lambda to understand its role in serverless computing, including how functions are structured, triggered, and integrated with AWS services. Learn about execution roles, environment variables, invocation types, versioning, and concurrency. This lesson prepares you to design scalable and secure serverless applications while managing Lambda's operational limits effectively.

In this lesson, we explore AWS Lambda service that abstracts infrastructure management and enables event-driven application development. We learn how Lambda functions are structured, triggered, invoked, versioned, and integrated with other AWS services. Building this foundational understanding prepares us to create scalable, cost-efficient, and resilient serverless applications.

Introduction to AWS Lambda

AWS Lambda is a serverless compute service that allows us to run code without provisioning or managing servers. It operates under the Function-as-a-Service (FaaS) model, where we provide the code, and AWS manages the underlying infrastructure.

Ask

In a serverless architecture, Lambda functions typically contain business logic and serve as the glue connecting multiple AWS-managed services. Users are charged based on the number of function invocations and the time it takes the code to execute, making Lambda a cost-efficient solution for scalable workloads.

How Lambda functions work

To interact consistently with different AWS services, Lambda functions follow a structured format. Each Lambda function operates using three key elements:

- **Invocation event:** The structured data that triggers the function. Its format varies based on the source service, such as API Gateway, S3, or DynamoDB Streams. For instance, an API Gateway event may include the HTTP method, query string parameters, headers, and body, while an S3 event contains the bucket name and object key.
- **Function handler:** The entry point within the function code that processes incoming events. This is a standard method written in the chosen programming language.
- **Context object:** Metadata about the function invocation, including function name, memory configuration, request ID, and the remaining execution time.

The code snippet below shows the default function handler in Python.

```python
def lambda_handler(event, context):
    # The function logic goes here
    return {
        'statusCode': 200,
        'body': 'Hello from Lambda!'
    }
```

lambda_handler.py

Ask

This handler receives two arguments, `event` and `context`, and returns a JSON-formatted HTTP response. When invoked, Lambda launches a container runtime

that initializes the function's environment, loads the code, and executes the handler. This environment may be reused across multiple invocations to optimize performance.

Understanding the function structure is the first step. Next, we examine how a Lambda function interacts securely with other AWS services through an execution role.

Lambda execution role

When a Lambda function needs to access other AWS services, it requires permissions to perform those actions securely. AWS Lambda achieves this by assuming an execution role—an AWS Identity and Access Management (IAM) role assigned to the function when it is created.

Lambda function accessing other AWS resources

The execution role defines what resources the function can interact with d
runtime. For example, if a function needs to read an object from an S3 bucket or write a record to a DynamoDB table, the permissions must be explicitly granted in

Ask

its execution role policy. Without proper permissions, Lambda invocations will fail with `AccessDenied` errors.

Managing execution roles carefully is critical for security and operational success. We should follow the principle of least privilege, granting only the permissions necessary for the function to operate as intended.

Environment variables

Lambda allows us to define *environment variables* that are key-value pairs accessible within the function code at runtime. These variables commonly store configuration values, such as database connection strings or third-party API keys.

For enhanced security, environment variables can be encrypted using AWS Key Management Service (KMS). Lambda automatically decrypts the environment variables during function initialization, making them available securely to the application code. Understanding configuration and concurrency controls prepares us to design scalable and secure serverless applications.

We've already discussed what happens when a Lambda function is triggered. Now, let's look at the different sources that can be used to trigger it.

Lambda event sources

Lambda functions are triggered by events generated by AWS services, custom applications, or direct HTTP requests. These event sources fall into two categories:

1. Direct triggers
2. Event source mappings

Understanding both helps us design integrations that are efficient, scalabl suited to our application's needs.

Ask

Direct triggers

Direct triggers include AWS services or features that invoke a Lambda function automatically in response to events. These services push events directly to Lambda, requiring no polling or intermediate configuration.

Common types of direct triggers include:

- **AWS service integrations:** Services such as Amazon S3, Amazon API Gateway, AWS EventBridge, and AWS Step Functions can invoke a Lambda function when predefined events occur. For example, a new object in an S3 bucket or an HTTP request to an API Gateway endpoint can trigger execution.
- **Lambda Function URLs:** Another way to invoke a Lambda function is a Function URL, an HTTP(S) endpoint. We can configure the function URL through the AWS management console or AWS CLI, which is a distinct URL associated with the function and remains constant. The typical structure of the function URL is given below:

```
https://<url-id>.lambda-url.<region>.on.aws
```

Function URL generic structure

Event source mappings

Some services do not push events directly to Lambda. Instead, Lambda must poll these services to detect new records or messages. This mechanism is known as event source mapping.

Event source mappings are commonly used with:

- Amazon DynamoDB Streams.
- Amazon Kinesis Data Streams.
- Amazon SQS.

Ask

Lambda polls these services at regular intervals, using permissions from the function's execution role. We can configure batch size, error handling, and concurrency behavior. For example, if records are added to a DynamoDB stream, the event source mapping automatically invokes the Lambda function with those records.

By understanding how each type of trigger operates, we can choose the right mechanism for their workload and control when and how their functions are invoked. Once event sources are configured, understanding the invocation behavior becomes critical.

Lambda invocation types

Depending on how quickly a response is needed, Lambda supports three primary invocation models:

- **Synchronous invocation:** The caller waits for the function's response. It's the default mode when invoking functions using the AWS CLI or SDKs like `boto3` or `aws-sdk`. This type of invocation is commonly used when testing or integrating Lambda into automation workflows, such as CI/CD pipelines. Here's how we can invoke a Lambda function synchronously using the CLI and SDK as follows:

AW...

```
aws lambda invoke \
  --function-name my-function \
  --payload '{"key": "value"}' \
  response.json
```

Ask

Lambda synchronous invocation using AWS CLI

These examples demonstrate how to invoke a Lambda function programmatically using different tools: the AWS CLI and Python's `boto3`. In each case, a sample JSON payload is passed to the function, and the result is captured or logged.

- **Asynchronous invocation:** In asynchronous invocation, the caller sends the event and receives a success message indicating that the event has been received. AWS Lambda then queues the event internally and invokes the function in the background. This pattern is ideal when immediate feedback to the caller is not required. We can also invoke functions asynchronously using the CLI or SDKs by setting the `InvocationType` parameter to `Event`:

AW...

```
aws lambda invoke \
  --function-name my-function \
  --invocation-type Event \
  --payload '{"key": "value"}' \
  response.json
```

Lambda asynchronous invocation using AWS SDK

Ask

- **Poll-based invocation:** Lambda polls a resource, such as a DynamoDB Stream or an SQS queue, and triggers function execution when new records arrive.

Selecting the correct invocation model is critical for balancing responsiveness, reliability, and system cost.

Lambda function destinations

For asynchronous workflows, such as those triggered by S3 or SNS, Lambda provides a structured way to handle the result of an invocation. These use cases do not return the event outcomes to the caller, so it is critical to manage them explicitly. Lambda supports a feature called destinations that defines where to send the result after the function finishes processing.

Supported destinations include:

- Another Lambda function.
- An S3 bucket.
- An SNS topic.
- An SQS queue.
- An EventBridge event bus.

Destinations allow us to handle successful and failed executions separately, enabling automatic retries, notification workflows, and fallback processing without complex custom logic inside the main function.

Validating function behavior

Now that we understand how Lambda invocation works, the next step is to learn how to test the Lambda code before connecting it to AWS services like API Gateway or S3. We can do this by simulating how an upstream service will trigger the Lambda function. This allows us to focus on writing and refining business logic without needing the full integration setup in place.

Ask

Here's a step-by-step process for validating Lambda function code:

1. Create a test payload

We start by creating a sample event in a file. This represents what the triggering service (like API Gateway or S3) would send to our Lambda function. Using the command below we save the expected JSON payload into a file named `event.json`:

main.sh Shell

```
echo '{"key": "value"}' > event.json
```

Create a JSON payload

This payload will be passed as value of a parameter to our Lambda function.

2. Invoke the function using the AWS CLI

Once we've created the expected payload file, we can simulate the function invocation using the `invoke` command:

C++ Ask

```
aws lambda invoke \
  --function-name my-function \
  --payload file://event.json \
  response.json
```

Invoke the Lambda function with simulated payload

This command does the following:

- Sends the contents of `event.json` as the event input
- Triggers the Lambda function
- Writes the output of the function to `response.json`

If the function executes successfully, the CLI returns metadata including the status code and request ID, while the actual result is saved in the `response.json` file.

3. Verify the output

Finally, we open `response.json` to confirm that the function returned the expected result.

This workflow is essential when building and debugging Lambda functions locally or in early stages of deployment. It helps ensure the handler logic is working as expected before plugging it into more complex, event-driven architectures.

Cold start in Lambda

When a Lambda function is invoked after a period of inactivity, AWS may r initialize a new runtime environment before executing the function. This delay is known as a *cold start*, typically adding 100 milliseconds to 1 second of latency.

Ask

After processing, AWS keeps the runtime warm for a non-deterministic period. Subsequent invocations benefit from a *warm start*, significantly reducing latency.

To reduce cold start frequency, developers sometimes schedule regular “ping” invocations using EventBridge rules. However, this technique is not guaranteed. For production-grade low-latency needs, AWS offers provisioned concurrency, which keeps a set number of runtime instances pre-initialized and ready to respond instantly.

In addition to provisioned concurrency, AWS Lambda also supports reserved concurrency. Reserved concurrency guarantees a set number of concurrent executions for a function and simultaneously limits scaling beyond the reserved amount. While it does not eliminate cold starts, reserved concurrency ensures that critical functions have available capacity and prevents them from being throttled when other functions consume shared account concurrency.

Lambda versioning and aliases

Effective deployment and traffic management rely on the ability to isolate and promote Lambda versions safely. AWS Lambda supports:

- **Versions:** To manage deployments safely, AWS Lambda lets us publish a version of a function. This captures the current state of the function's code and configuration as a read-only snapshot that won’t change, even if the function is updated later. Versioning is useful when promoting code to staging or production, especially in workflows that use CI/CD.
- **Aliases:** These are named references that point to specific versions. Aliases allow us to deploy updates gradually, manage environments like `dev`, `test`, and `prod`, and perform blue/green deployments or canary releases.

By default, every update to a Lambda function modifies the unpublished `$` version. These changes are not versioned automatically. To make a deployment reproducible or stable, we must explicitly publish a version.

Ask

To publish a version using the AWS CLI, we can use the following command:

AW...

```
aws lambda publish-version \
  --function-name my-function
```

Publish a Lambda version using AWS CLI

This creates a new, numbered version (like, `1`, `2`, `3`) based on the current `$LATEST` code and configuration.

Once we've published a version, we can create an alias pointing to that specific version using the following command:

AW...

```
aws lambda create-alias \
  --function-name my-function \
  --name prod \
  --function-version 3
```

Ask

Create an alias using AWS CLI

After we've created an alias, `prod` in the code above, we can invoke our function through the alias instead of using the `$LATEST` or version number. The command below invokes version `3` of the Lambda function:

AW...

```
aws lambda invoke \
  --function-name my-function:prod \
  --payload '{"key": "value"}' \
  response.json
```

Invoke a specific version of a Lambda function using AWS CLI

`my-function:prod` in **line 2** tells Lambda to invoke the `my-function` Lambda function using the code and configuration from the version the `prod` alias points to.

By associating provisioned concurrency with aliases, we can ensure that only stable, production-ready versions benefit from pre-initialized runtime instances

Ask

Lambda limits to remember

Several key operational limits apply to AWS Lambda:

- **Maximum execution timeout:** 900 seconds (15 minutes) per invocation. Workloads requiring longer processing must use AWS Step Functions or other services.
- **Deployment package size:** Up to 50 MB compressed (direct upload) or 250 MB uncompressed (including dependencies).
- **Ephemeral disk space:** Up to 10 GB in the `/tmp` directory, configurable.
- **Concurrent executions:** Soft limit of 1000 concurrent executions per AWS Region per account (can be increased upon request).
- **Environment variable size:** Up to 4 KB total for all environment variables combined.

Being mindful of these limits helps ensure reliable, scalable applications that operate efficiently within Lambda's constraints.

Conclusion

Serverless computing changes how we build and run applications by removing the need to manage infrastructure. With AWS Lambda, functions become the core unit of execution, short-lived, event-driven, and inherently scalable. Success depends on understanding how functions are triggered, how permissions are configured, and how code is packaged and deployed. Architecting for this model means thinking in terms of stateless execution, efficient cold starts, and tight integration with the broader AWS ecosystem. When done right, it enables faster development cycles, reduced operational burden, and a system that adapts

Ask